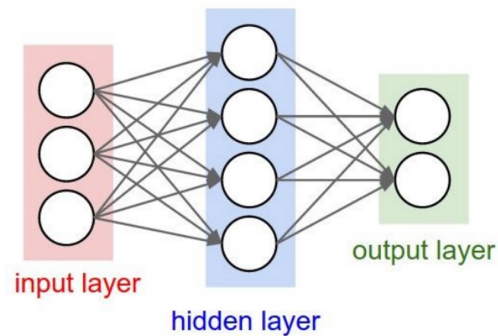


Atomic Units of Language

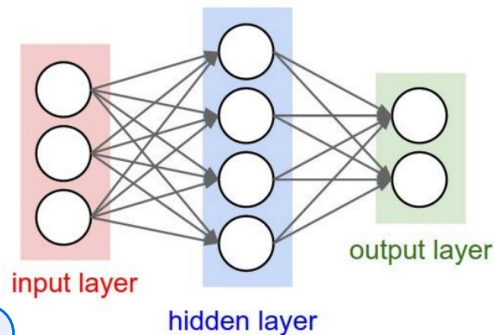
CSCI 601-471/671 (NLP: Self-Supervised Models)

Recap: Neural Nets

- A powerful function-approximation tool.
- Can be trained efficiently via Backpropagation.
- Our focus here: how to use NNs for language modeling.

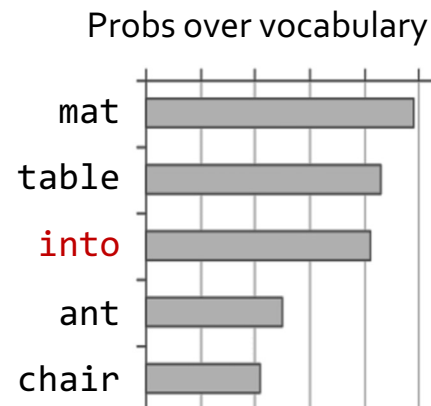
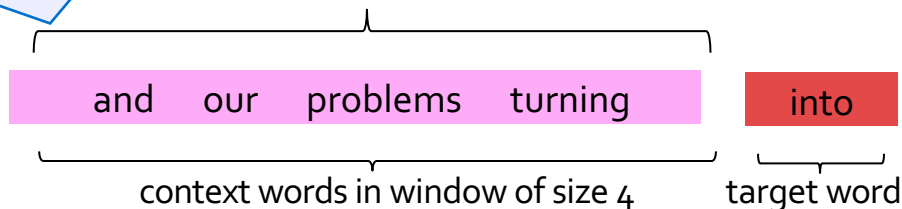


Big Picture: Language Modeling + NNs



Remember NNs expect numbers. How do you feed a string to neural net?

$$f(\mathbf{x}, \Theta)$$



Atomic Units of Language

The cat sat on the mat.

Which one should we use as **the atomic building blocks** for modeling language? 🤔

- words split based on white space?

BOS, The, cat, sat, on, the, mat., EOS

- characters?

BOS, T, h, e, SPACE, c, a, t, SPACE, s, ...

- bits??!

01010100 01101000 01100101 00100000
01100011 01100001 01110100 ...

Cost of Using Word Units

- What happens when we encounter a word at test time that **we've never seen in our training data**?
 - *Omnishambles*: A situation that has been mismanaged, due to blunders and miscalculations.
 - *COVID-19*: was unseen until 2020!
- What about related words?: “dog” vs “dogs”; “run” vs “running”
- We would need a **very large** vocabulary to capture common words in a language.
 - Very large vocabulary size makes training difficult
- What happens with words that we haven't seen before?
 - With **word level** tokenization, unseen words map to [UNK], **losing their identity**.
 - Also, not all languages have spaces between words like English!

Cost of Using Character Units

- What if we use **characters**?

- Pro:**

- (1) **small vocabulary**, just the number of unique characters in the training data.
- (2) fewer out-of-vocabulary tokens

- Cost:** **much longer input sequences**

- As we discussed, modeling long-range dependences is very challenging.
- Representing long sequences is computationally costly.

a	→	1
b	→	2
c	→	3
d	→	4
e	→	5
f	→	6
g	→	7
...	→	...
1	→	27
2	→	28
3	→	29
...	→	...
!	→	37
...	→	...
à	→	256

Characters as tokens

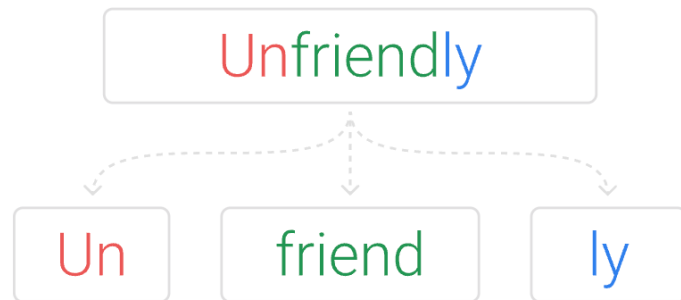
the	→	1
of	→	2
and	→	3
to	→	4
in	→	5
was	→	6
the	→	7
is	→	8
for	→	9
as	→	10
on	→	11
with	→	12
that	→	13

...	→	...
malapropism	→	170,000

words as tokens

Subword Tokenization: A Middle Ground

- Learn reusable text pieces from a corpus; these *sometimes align* with morphemes
- Subword tokenization is a practical trade-off.
 - Common words are preserved in the vocabulary.
 - Less common words are broken down into sub-words.
 - This handles the problem of unseen words and large vocabulary size.
- Dominantly used in modern language models.



Trying an actual tokenizer

```
from transformers import AutoTokenizer

text = "The unbelievably fluffy cat purred."

for model in ["Qwen/Qwen3-0.6B", "deepseek-ai/DeepSeek-V3"]:
    tokenizer = AutoTokenizer.from_pretrained(model)

    ids = tokenizer.encode(text, add_special_tokens=False)
    tokens = tokenizer.convert_ids_to_tokens(ids)

    print(model)
    print(tokens)
    print(ids)
```

Trying an actual tokenizer

- Output for Qwen3: 8 tokens:

```
['The', 'Ġunbelie', 'vably', 'Ġfluffy', 'Ġcat', 'Ġp', 'urred', '.']  
[785, 38937, 88134, 67561, 8251, 281, 8130, 13]
```

- Output for DeepSeek-V3:

```
['The', 'Ġun', 'bel', 'iev', 'ably', 'Ġfluffy', 'Ġcat', 'Ġpur', 'red', '.']  
[671, 612, 12870, 41607, 3188, 81340, 7539, 2805, 1157, 16]
```

```
from transformers import AutoTokenizer  
  
text = "The unbelievably fluffy cat purred."  
  
for model in ["Qwen/Qwen3-0.6B", "deepseek-ai/DeepSeek-V3"]:  
    tokenizer = AutoTokenizer.from_pretrained(model)  
  
    ids = tokenizer.encode(text, add_special_tokens=False)  
    tokens = tokenizer.convert_ids_to_tokens(ids)  
  
    print(model)  
    print(tokens)  
    print(ids)
```

Ġ as the
representation of space

Trying OpenAI's Tokenizer

- How many tokens do we have here? (note, there are two spaces before the number.)
 - Where would you put the token boundaries?
What happens to the unfamiliar word, the spaces, and the number?

The cat saw a quokka 12345.

The cat saw a quokka 12345.

Frequent words are
one token

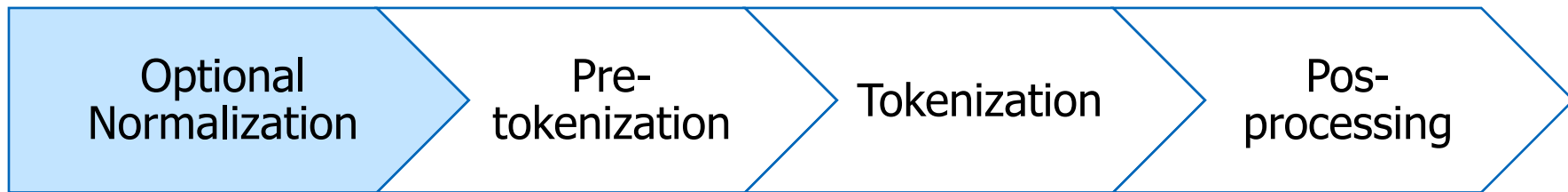
Rare word split into
multiple tokens

Large number split
into multiple tokens

Space before "cat" is part of
the token. So, the tokenizer
probably has two cat tokens:
w/ and w/o space.

Extra space
is counted
toward extra
tokens

The Tokenization Pipeline



- Converts text into a standard format to reduce variability.
 - Stripping extra white spaces between words and sentences
 - Removing punctuations (“Hello, world!” → “Hello world”)
 - Unicode normalization (more on this in a sec)
 - ...
- Note that these are **optional**.
 - While they reduce variability, they’re destroy information (e.g., spacing information for code) and prevent fully faithful reconstruction.
 - Most modern LLMs don’t do these.

Unicode normalization

- There are many characters that look very similar (or identical) but are different in Unicode:

Roman "A"	A	U+0041
Fullwidth "A"	A	U+FF21

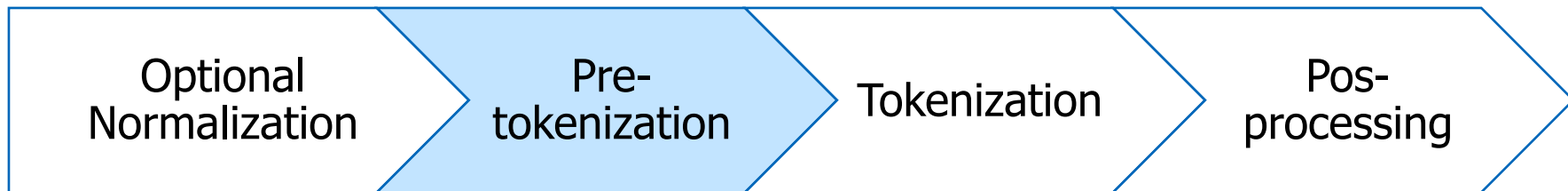
- There are various normalization forms to map equivalent Unicode representations (fullwidth vs ASCII, ligatures, old typographic variants) to a common character sequence.

Positional variant forms	ε	→	ε
	ε	→	ε
	ε	→	ε
	ε	→	ε

Circled variants	①	→	1
Width variants	力	→	力
Fractions	¼	→	1/4

- NFKC normalization is a double-edged sword:
 - Benefit:** shrinks vocabs size (and more compact model)
 - Cost:** it's lossy and not reversible. After NFKC, we cannot tell whether A originally came from 'A' or 'A'.

The Tokenization Pipeline

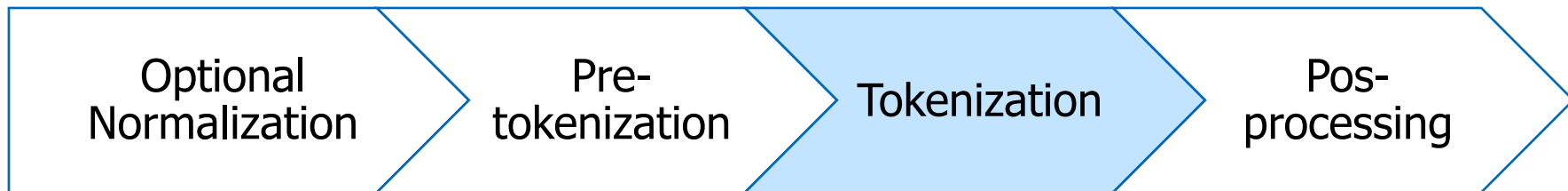


- A pre-tokenizer splits text into chunks. (often regex- or rule-based).
 - The subsequent tokenization (e.g., BPE) operates independently inside each chunk.

The cat saw a quokka 12345.

- Chunks: [The] [·cat] [·saw] [·a] [·quokka] [·] [·] [123] [45] [·.]
- Tokenization (e.g., BPE) can merge pieces within a chunk but cannot merge across pre-tokenization boundaries.
- **Note:** The chunking rules here (attaching leading spaces to words and grouping digits into chunks of up to three) are tokenizer-specific.

The Tokenization Pipeline



- **Training time:** Train the tokenizer on the pre-tokenized text (forthcoming)
 - Not to confuse this with LLM training, which is done after deciding your tokenizer.
- **Inference time:** Apply the trained tokenizer to each chunk.

The Tokenization Pipeline



- Truncate to match the maximum length of the model
- Pad all sentences in a batch to the same length
- Add special tokens: for example:
 - **<UNK>** (unknown word; when an item cannot be represented with your tokenizer)
 - **<PAD>** (padding for fixed-length sequences)
 - **<BOS>** (to indicate the beginning of sentence)
 - **<EOS>** (to indicate the end of sentence)
- **Finally**, converts tokens into numerical IDs for model input.
 - Uses a vocabulary (word-to-index mapping).

Byte-pair Encoding (BPE) – An Example

- An algorithm for **forming subword** tokens based on **a collection of raw text**.

and there are no refueling stations anywhere
one of the city's more unprincipled real estate agents

Note that to do this
tokenization, I need to learn it
by seeing lots of text. (similar
to model training)

*BPE-based
tokenization*

[and] [·there] [·are] [·no] [·re] [fueling] [·stations]
[·anywhere] [one] [·of] [·the] [·city] ['s] [·more] [·un]
[princi] [pled] [·real] [·estate] [·agents]

Byte-pair Encoding (BPE) Training

Overview:

- We are given a **large** text corpus of text.
- Start by character-based tokenization.
- Repeatedly merge the most frequent adjacent tokens
- **Vocabulary size** := initial alphabet + new merge tokens + special tokens.
 - Doing 30k merges $\rightarrow$ vocab of a > 30k subwords.

Algorithm 1 Byte-pair encoding (Sennrich et al., 2016; Gage, 1994)

```
1: Input: set of strings  $D$ , target vocab size  $k$ 
2: procedure BPE( $D, k$ )
3:    $V \leftarrow$  all unique characters in  $D$ 
4:   (about 4,000 in English Wikipedia)
5:   while  $|V| < k$  do ▷ Merge tokens
6:      $t_L, t_R \leftarrow$  Most frequent bigram in  $D$ 
7:      $t_{\text{NEW}} \leftarrow t_L + t_R$  ▷ Make new token
8:      $V \leftarrow V + [t_{\text{NEW}}]$ 
9:     Replace each occurrence of  $t_L, t_R$  in
10:       $D$  with  $t_{\text{NEW}}$ 
11:   end while
12:   return  $V$ 
13: end procedure
```

BPE training: Example

- *Example:*

Our (very fascinating 🤪) training data: “jhu jhu jhu hopkins hop hops hops”

Pre-tokenize by attaching each space to the following word:

[jhu] [·jhu] [·jhu] [·hopkins] [·hop] [·hops] [·hops]

Base vocab: ·, h, i, j, k, n, o, p, s, u

Tokenized data: [j h u] [· j h u] [· j h u] [· h o p k i n s] [· h o p] [· h o p s] [· h o p s]

BPE training: Example

- *Example:*

Our (very fascinating 🤔) training data: “jhu jhu jhu hopkins hop hops hops”

Pre-tokenize by attaching each space to the following word:

[jhu] [·jhu] [·jhu] [·hopkins] [·hop] [·hops] [·hops]

Base vocab: ·, h, i, j, k, n, o, p, s, u

Tokenized data: [j h u] [· j h u] [· j h u] [· h o p k i n s] [· h o p] [· h o p s] [· h o p s]

- $\cdot + h \rightarrow 4$ $h + o \rightarrow 4$ $o + p \rightarrow 4$
- $j + h \rightarrow 3$ $h + u \rightarrow 3$
- $\cdot + j \rightarrow 2$ $p + s \rightarrow 2$
- $p + k \rightarrow 1$ $k + i \rightarrow 1$ $i + n \rightarrow 1$ $n + s \rightarrow 1....$

Count adjacent
token pairs

BPE training: Example

- *Example:*

Our (very fascinating 🤔) training data: “jhu jhu jhu hopkins hop hops hops”

Pre-tokenize by attaching each space to the following word:

[jhu] [·jhu] [·jhu] [·hopkins] [·hop] [·hops] [·hops]

Base vocab: ·, h, i, j, k, n, o, p, s, u

Tokenized data: [j h u] [· j h u] [· j h u] [· h o p k i n s] [· h o p] [· h o p s] [· h o p s]

- $\cdot + h \rightarrow 4$ $h + o \rightarrow 4$ $o + p \rightarrow 4$
- $j + h \rightarrow 3$ $h + u \rightarrow 3$
- $\cdot + j \rightarrow 2$ $p + s \rightarrow 2$
- $p + k \rightarrow 1$ $k + i \rightarrow 1$ $i + n \rightarrow 1$ $n + s \rightarrow 1....$

Choose the most frequent pair

BPE training: Example

- *Example:*

Our (very fascinating 🤔) training data: “jhu jhu jhu hopkins hop hops hops”

Pre-tokenize by attaching each space to the following word:

[jhu] [·jhu] [·jhu] [·hopkins] [·hop] [·hops] [·hops]

Base vocab: ·, h, i, j, k, n, o, p, s, u, ho ←

Tokenized data: [j h u] [· j h u] [· j h u] [· h o p k i n s] [· h o p] [· h o p s] [· h o p s]

- $\cdot + h \rightarrow 4$ $h + o \rightarrow 4$ $o + p \rightarrow 4$
- $j + h \rightarrow 3$ $h + u \rightarrow 3$
- $\cdot + j \rightarrow 2$ $p + s \rightarrow 2$
- $p + k \rightarrow 1$ $k + i \rightarrow 1$ $i + n \rightarrow 1$ $n + s \rightarrow 1....$

Add the merged token to the vocabulary

BPE training: Example

- Example:

Our (very fascinating 🤔) training data: “jhu jhu jhu hopkins hop hops hops”

Pre-tokenize by attaching each space to the following word:

[jhu] [·jhu] [·jhu] [·hopkins] [·hop] [·hops] [·hops]

Base vocab: ·, h, i, j, k, n, o, p, s, u, ho

Tokenized data: [j h u] [· j h u] [· j h u] [· ho p k i n s] [· ho p] [· ho p s] [· ho p s] ←

- $\cdot + h \rightarrow 4$ $h + o \rightarrow 4$ $o + p \rightarrow 4$
- $j + h \rightarrow 3$ $h + u \rightarrow 3$
- $\cdot + j \rightarrow 2$ $p + s \rightarrow 2$
- $p + k \rightarrow 1$ $k + i \rightarrow 1$ $i + n \rightarrow 1$ $n + s \rightarrow 1....$

Apply the merge to the
tokenize sequence:
Replace every within-
chunk occurrence of $h + o$

BPE training: Example

- *Example:*

Our (very fascinating 🤔) training data: “jhu jhu jhu hopkins hop hops hops”

Pre-tokenize by attaching each space to the following word:

[jhu] [·jhu] [·jhu] [·hopkins] [·hop] [·hops] [·hops]

Base vocab: ·, h, i, j, k, n, o, p, s, u, ho

Tokenized data: [j h u] [· j h u] [· j h u] [· ho p k i n s] [· ho p] [· ho p s] [· ho p s]

- · + ho → 4 ho + p → 4
- j + h → 3 h + u → 3
- · + j → 2 p + s → 2
- p + k → 1 k + i → 1 i + n → 1 n + s → 1

Recount adjacent
token pairs

BPE training: Example

- *Example:*

Our (very fascinating 🤔) training data: “jhu jhu jhu hopkins hop hops hops”

Pre-tokenize by attaching each space to the following word:

[jhu] [·jhu] [·jhu] [·hopkins] [·hop] [·hops] [·hops]

Base vocab: ·, h, i, j, k, n, o, p, s, u, ho

Tokenized data: [j h u] [· j h u] [· j h u] [· ho p k i n s] [· ho p] [· ho p s] [· ho p s]

- · + ho → 4 **ho + p → 4**
- j + h → 3 h + u → 3
- · + j → 2 p + s → 2
- p + k → 1 k + i → 1 i + n → 1 n + s → 1

Choose a
new pair

BPE training: Example

- *Example:*

Our (very fascinating 🤔) training data: “jhu jhu jhu hopkins hop hops hops”

Pre-tokenize by attaching each space to the following word:

[jhu] [·jhu] [·jhu] [·hopkins] [·hop] [·hops] [·hops]

Base vocab: ·, h, i, j, k, n, o, p, s, u, ho, hop ←

Tokenized data: [j h u] [· j h u] [· j h u] [· ho p k i n s] [· ho p] [· ho p s] [· ho p s]

- · + ho → 4 **ho + p → 4**
- j + h → 3 h + u → 3
- · + j → 2 p + s → 2
- p + k → 1 k + i → 1 i + n → 1 n + s → 1

Add the new merged token to our vocab

BPE training: Example

- Example:

Our (very fascinating 🤪) training data: “jhu jhu jhu hopkins hop hops hops”

Pre-tokenize by attaching each space to the following word:

[jhu] [·jhu] [·jhu] [·hopkins] [·hop] [·hops] [·hops]

Base vocab: ·, h, i, j, k, n, o, p, s, u, ho, hop

Tokenized data: [j h u] [· j h u] [· j h u] [· hop k i n s] [· hop] [· hop s] [· hop s] ←

- · + ho → 4 ho + p → 4
- j + h → 3 h + u → 3
- · + j → 2 p + s → 2
- p + k → 1 k + i → 1 i + n → 1 n + s → 1

Add the new merged token to our vocab

Note: Training learns the vocabulary and merge order.
Inference applies them!

BPE at inference time

- Start with new text and apply the same normalization + pre-tokenization used during training.
- Split each chunk into base units (e.g., characters).
- Within each chunk, repeatedly apply the **earliest-learned** merge that is currently possible.
- Stop when no learned merge applies, then map the resulting tokens to IDs.

BPE at inference time: example

- Suppose training learned these rules, in order:
 - 1. l + o $\rightarrow$ lo
 - 2. lo + w $\rightarrow$ low
 - 3. e + r $\rightarrow$ er
 - 4. low + er $\rightarrow$ lower
- Now we want to tokenize: “lowered”
 - [l o w e r e d]
 - $\downarrow$ Rule 1: l + o $\rightarrow$ lo
 - [lo w e r e d]
 - $\downarrow$ Rules 2 and 3 both apply; rule 2 has priority
 - [low e r e d]
 - $\downarrow$ Rule 3: e + r $\rightarrow$ er
 - [low er e d]
 - $\downarrow$ Rule 4: low + er $\rightarrow$ lower
 - [lower e d]

What is the computational complexity of doing this?

BPE at inference time—efficiently

- For a given chunk, we look up the relevant merge rules in a hash table:
 $(\text{left_token_id}, \text{right_token_id}) \rightarrow (\text{merged_token_id}, \text{merge_rank})$
- For example, if our chunk is [a b c d], the relevant rules will be:
 - (a, b) $\rightarrow$ rank 800
 - (b, c) $\rightarrow$ no learned merge
 - (c, d) $\rightarrow$ rank 12,000
- A lookup takes expected $O(1)$ time, regardless of vocabulary size V .
- For a chunk starting with n tokens:
 - There are at most $n-1$ merges.
 - Each merge introduces at most two new neighboring pairs to check.
 - Managing those candidates in the priority queue costs $O(\log n)$ per operation.
 - That gives $O(n \log n)$ expected time per chunk.
- Vocab size still affects storage and initialization: the vocabulary and merge table typically occupy $O(V)$ space and are loaded once, then reused across inputs.

Pulse check: True or false

- **Q:** BPE always splits words into meaningful linguistic units.
- **A: False:** merges reflect frequency, not necessarily meaning.

- **Q:** A word absent from the tokenizer's training data must become <UNK>.
- **A: False:** an unseen word can be composed of known smaller tokens. An unsupported base character may still cause <UNK> without byte fallback.

- **Q:** At inference time, BPE chooses merges by counting pair frequencies in the new input.
- **A: False:** inference uses the learned merge priorities.

- **Q:** Increasing the BPE vocabulary usually trades shorter sequences for larger embedding and output matrices.
- **A: True:** this is a central vocabulary-size trade-off.

Limitations of BPE

- The base vocabulary must include every character you want to handle.
 - Unicode has ~150k characters. Including them all takes up the vocabulary.
- Token boundaries need not correspond to meaningful linguistic units.
 - E.g., “Understand” -> [“Under”, “stand”] -Doesn’t mean “stand beneath”!
- Contiguous subwords cannot capture every kind of language morphology.
 - In languages with **non-concatenative** (e.g., Arabic) morphology, roots and inflection are not separable via contiguous split of subwords.

كتب	k-t-b	“write” (root form)
كَتَبَ	kataba	“he wrote”
كَتَبَ	kattaba	“he made (someone) write”
اِكتَتَبَ	iktataba	“he signed up”

Table 1: Non-concatenative morphology in Arabic.³
When conjugating, letters are interleaved *within* the root. The root is therefore not separable from its inflection via any contiguous split.

BPE is the dominant algorithm.

Most BPE is based on Bytes.
We rarely use character-based BPE.

Model	Year	Algorithm (library)	Base unit
Qwen3 / Qwen3.5 / Qwen3.6	2025–26	BPE (HF tokenizers)	Byte-level
Gemma 1 / 2 / 3 / 3n / 4	2024–26	BPE (SentencePiece)	Characters + byte fallback
GLM-4.5 / GLM-5 / 5.1 / 5.2	2025–26	BPE (HF tokenizers)	Byte-level
Kimi K2 / K2.5 / K2.6 / K3	2025–26	BPE (tiktoken)	Byte-level
DeepSeek-V3 / R1 / V4	2024–26	BPE (HF tokenizers)	Byte-level
Mistral 2 / 3 / 4	2024–26	BPE (tiktoken)	Byte-level
GPT-5 / GPT-4.1 / GPT-4o / GPT-OSS	2024–25	BPE (tiktoken)	Byte-level
Llama 3 / 3.1 / 3.3	2024	BPE (tiktoken)	Byte-level
GPT-4 / GPT-3.5	2022–23	BPE (tiktoken)	Byte-level
Mistral 7B / Mixtral	2023	BPE (SentencePiece)	Characters + byte fallback
Llama 2	2023	BPE (SentencePiece)	Characters + byte fallback
T5	2019–22	Unigram (SentencePiece)	Characters only (<unk> possible)
BERT	2018	WordPiece	Characters only (<unk> possible)

Alternatives: Byte-level BPE

- **Start from bytes, not characters.** Text gets encoded as a sequence of UTF-8 bytes, so the base vocabulary is the $2^8 = 256$ possible byte values.
 - ASCII characters are 1 byte (e.g., H $\rightarrow$ 01001000, value: 72).
 - Other characters take 2-4 bytes (é $\rightarrow$ C3 A9, 😊 $\rightarrow$ F0 9F 98 80).
- **Pre-tokenize first.** A regex splits text into chunks (words, numbers, punctuation, whitespace), and merges happen only within a chunk. (same as before)
- **Train BPE on the byte sequences.** Repeatedly merge the most frequent adjacent pairs.
 - Early merges often reassemble multi-byte characters;
 - Later ones build subwords and whole words.
- Detokenize (e.g., at generation time) by concatenating the tokens' bytes, then UTF-8 decoding.
- **Pros:** (1) no out-of-vocabulary problem ever; (2) no need for a huge base vocabulary.

BPE + Character fallback

- **Base vocabulary = frequent characters + 256 byte tokens.**
 - Keep the most frequent characters that cover a target corpus (avoid char blowup).
 - Add reserved byte tokens <0x00> ... <0xFF>.
- **Train BPE merges over characters** (as in character-level BPE).
 - Byte tokens never take part in merges.
- **At encoding time, an unknown character becomes its UTF-8 bytes** instead of [UNK].
 - E.g., "hello 🦙" → [_hello, _, <0xF0>, <0x9F>, <0xA6>, <0x99>]
- **Pros:**
 - No out-of-vocabulary problem, as in byte-level BPE.
 - Normal tokens contain whole characters, so they never split a character.
- **Cons:**
 - Chars outside the vocabulary always cost 2-4 tokens, since byte tokens are never merged.

Alternatives: WordPiece

- Builds a vocabulary to improve the likelihood of the training corpus.
 - **Training recipe***: Start with chars; repeat until the target vocabulary size:
 - While vocab size < target:
 1. Count tokens and adjacent pairs.
 2. Select the pair with the highest score:
 3. Merge the pair, add the new token, and update counts.
- $$\text{score}(a, b) = \frac{\text{freq}(ab)}{\text{freq}(a) \text{freq}(b)}$$

- **Intuition**: Favors pairs that occur together frequently relative to their individual frequencies.
 - ("un", "##able") may not merge despite its high frequency, as both parts appear frequently in corpus.
 - ("hu", "##gging") may merge faster if "hugging" is common, as its parts are individually less frequent.

*(*Illustrative reconstruction; Google's original vocabulary-training implementation was not released.)*

Alternatives: Unigram LM

- Starts with a large vocabulary of candidate substrings and **prunes it down**.
 - BPE/WordPiece: **grow** the vocabulary by merging.
 - Unigram LM: **shrink** the vocabulary by removing tokens.
- A probabilistic problem: The best tokenization is the segmentation with the highest probability.
- **Training:**
 - Start with a large candidate vocabulary, such as all frequent substrings.
 - Repeat until the target vocabulary size
 - Use EM to estimate each subword's probs.
 - For each subword, compute how much the corpus likelihood drops if it were removed.
 - Prune the lowest-impact tokens, e.g., the bottom 10–20%. (Retain chars for coverage)
- **Inference:** is done via **dynamic programming (Viterbi)** to find the most prob segmentation.

Alternatives: Unigram LM

Original: furiously

BPE: _fur iously

Uni. LM: _fur ious ly

Original: tricycles

BPE: _t ric y cles

Uni. LM: _tri cycle s

Original: nanotechnology

BPE: _n an ote chn ology

Uni. LM: _nano technology

Original: Completely preposterous suggestions

BPE: _Comple t ely _prep ost erous _suggest ions

Unigram LM: _Complete ly _pre post er ous _suggestion s

Original: corrupted

BPE: _cor rupted

Unigram LM: _corrupt ed

Original: 1848 and 1852,

BPE: _184 8 _and _185 2,

Unigram LM: _1848 _and _1852 ,

- Some have argued that BPE produces tokens that are less aligned with morphology.
- But BPE based LMs do work fine – linguistic alignment is one consideration alongside compression and downstream performance.

Dealing with white spaces

- **Why Whitespace Isn't Just "Stripped Out"**
 - Whitespace are not stripped.
 - Goal: reversibility —tokenizers must reconstruct the original text, spacing included.
- **BPE:** Often attaches space as a *prefix* to the next word ("world" vs. " world" are **different** tokens);
 - GPT-2 maps space to a visible symbol "Ġ" for uniform byte-level processing.
- The issue w/ these is long spaces since they're never merged into longer tokens.
- GPT-4's fix (cl100k_base): introduced a new pre-tokenization regex and added dedicated multi-space tokens. This groups 4 spaces into a single token and has dedicated tokens for whitespace seqs up to 128 spaces.

GPT-2

```
def fibRec(n):↵  
    if n < 2:↵  
        return n↵  
    else:↵  
        return fibRec(n-1) + fibRec(n-2)
```

55 tokens

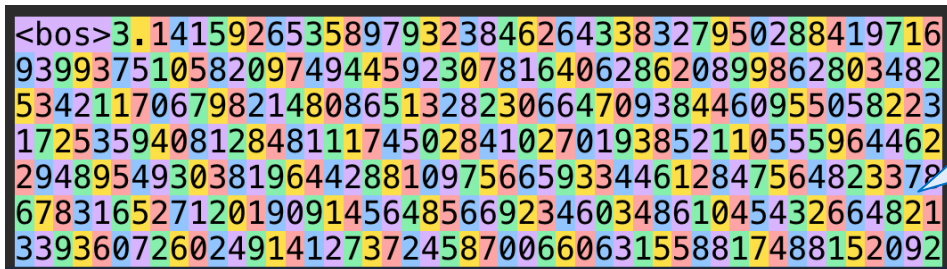
GPT-NeoX-20B

```
def fibRec(n):↵  
    if n < 2:↵  
        return n↵  
    else:↵  
        return fibRec(n-1) + fibRec(n-2)
```

39 tokens

Dealing with numbers

- Numbers are challenging:
 - BPE merges tokens based on frequency; digits frequency may be arbitrary.
 - Inconsistent tokenization depending on the context:
 - "1234" → 123+4, but "1235" → 12+35
 - "42" → 42 (single token; common token); "47" → 4+7
- **Fix #1:** Force every digit (0–9) to be its own token
 - Guarantees consistency: every "7" is always the same token
 - Trade-off: longer sequences for large numbers

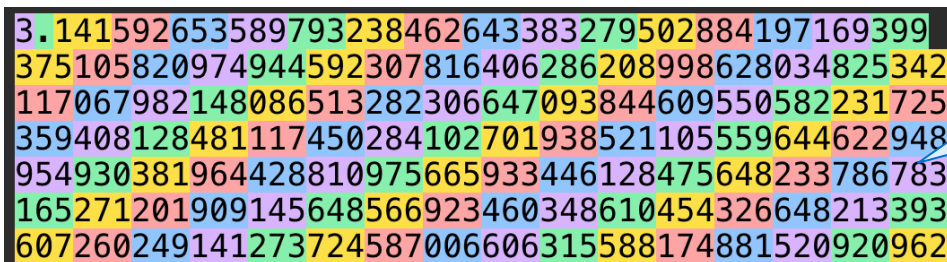


<bos>3.14159265358979323846264338327950288419716939937510582097494459230781640628620899862803482534211706798214808651328230664709384460955058223172535940812848111745028410270193852110555964462294895493038196442881097566593344612847564823378678316527120190914564856692346034861045432664821339360726024914127372458700660631558817488152092

Mixtral, Llama, DeepSeek, and Gemma tokenizers broke down numerical sequences into a separate token for each digit.

Dealing with numbers

- Numbers are challenging:
 - BPE merges tokens based on frequency; digits frequency may be arbitrary.
 - Inconsistent tokenization depending on the context:
 - "1234" $\rightarrow$ 123+4, but "1235" $\rightarrow$ 12+35
 - "42" $\rightarrow$ 42 (single token; common token); "47" $\rightarrow$ 4+7
- Fix #1:** Force every digit (0–9) to be its own token
 - Guarantees consistency: every "7" is always the same token
 - Trade-off: longer sequences for large numbers
- Fix #2:** Let BPE do its merging on numbers, but cap its length (commonly 3).



3.141592653589793238462643383279502884197169399
375105820974944592307816406286208998628034825342
117067982148086513282306647093844609550582231725
359408128481117450284102701938521105559644622948
954930381964428810975665933446128475648233786783
165271201909145648566923460348610454326648213393
607260249141273724587006606315588174881520920962

GPT-4 and **GPT-4o** tokenizers broke down numerical sequences into groups of 3.

Dealing with numbers

- Numbers are challenging:
 - BPE merges tokens based on frequency; digits frequency may be arbitrary.
 - Inconsistent tokenization depending on the context:
 - "1234" $\rightarrow$ 123+4, but "1235" $\rightarrow$ 12+35
 - "42" $\rightarrow$ 42 (single token; common token); "47" $\rightarrow$ 4+7
- **Fix #1:** Force every digit (0–9) to be its own token
 - Guarantees consistency: every "7" is always the same token
 - Trade-off: longer sequences for large numbers
- **Fix #2:** Let BPE do its merging on numbers, but cap its length (commonly 3).
- **Fix #3:** Right-to-left chunking are shown to be more effective.
 - Chunking digits from the left doesn't align with base-10 place value
 - "1234567" $\rightarrow$ 123 | 456 | 7 (left; misaligned) vs 1 | 234 | 567

Special tokens

- Special tokens are **manually** inserted into the vocab by model developers.
- These are **not** meant to be used or shown by human users. They're parsed by the system that is running the LM. Hence, they use strings that're unlikely to occur naturally.
- Examples from GPT-OSS:
 - `<|start|>` / `<|end|>` : mark the **start/end of a message**
 - `<|message|>` : separates a message's **header (role) from its content**
 - `<|channel|>` : labels **which output stream** a piece of text belongs to (reasoning vs. final answer)
 - `<|constrain|>` : specifies the **expected format for tool-call arguments**
 - `<|call|>` : marks the model **invoking a tool**
 - `<|return|>` : marks the **final end of a response**

See OpenAI's special tokens:

https://github.com/openai/tiktoken/blob/08a5f3b2c987ada4fc5aa1f16c643c203fa8acaa/tiktoken_ext/openai_public.py#L123-L144
<https://developers.openai.com/cookbook/articles/openai-harmony#special-tokens>

Special tokens

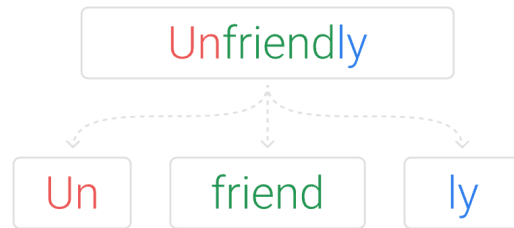
- Most tokenizers reserve a block of IDs up front for special tokens (e.g., Qwen reserves 208 extra empty slots).
- So new ones can be added later without resizing the whole vocabulary/embedding matrix.

Model	Release	Reserved block
Kimi K2, K2.5, K2.6	2025-26	3840
Qwen 2.5/3	2024-25	208
Gemma 3	2025	99
Llama 3	2024	256

Summary

- **Question:** what is a good atomic unit for feeding lang to LMs?

- **Words:** too coarse
- **Characters:** too small
- **Subwords:** a good middle ground.



- **One nice intuition:** tokenization is compression for [almost] free.

What Tokenizers do people use?

Model	Release	Algorithm	Library
Kimi K2, K2.5, K2.6	2025-26	Byte-level BPE	tiktokens
GLM-4/GLM-4.6	2024-25	Byte-level BPE	HF tokenizers
GPT-OSS	2025	Byte-level BPE	tiktokens
Qwen 2.5/3	2024-25	Byte-level BPE	tiktokens
Gemma 3	2025	Unigram LM	SentencePieces
DeepSeek-V3	2024	Byte-level BPE	HF tokenizers

- Nearly everyone (except Google) uses BPE.
- All these tokenizers are *mostly* invertible* (*except the ones that do aggressive normalization)

Vocab sizes

Model	Release	Vocab size
Kimi K2, K2.5, K2.6	2025-26	160k
GLM-4/GLM-4.6	2024-25	150k
GPT-OSS	2025	201k
Qwen 2.5/3	2024-25	151k
Gemma 3	2025	262k
DeepSeek-V3	2024	128k
mT5	2020	250k

Model	Release	Vocab size
Original Transformer	2017	37k
GPT1	2018	40k
GPT2/3	2019-20	50.2k
T5	2019	32k
Llama	2023	32k
GPT4	2023	100.2k

- Vocab size of English-only models (**right**) had 30-60k.
- While multi-lingual models (**left**) typically have >> 100k tokens.
- Some teams size vocabs to align with hardware/tensor-parallel efficiency.
 - $2^{15} = 32,768$; $2^{16} = 65,536$; $131,072 = 2^{17}$; $262,144 = 2^{18}$;

Summary so far

- Everyone uses invertible subword tokenizers (BPE, Unigram) for good reason.
- What are some new ideas in tokenization?

SuperBPE

- The standard BPE first splits the words into chunks separated by white spaces. So that a token in the vocab can *never cross the white space border*.
- What if tokens contained whitespace?
 - Multi-word expressions act as single semantic units (e.g., "by the way", "for example")
 - Longer tokens means model can generate more efficiently.
- **Approach:**
 - **Stage 1:** learn ordinary subwords (like normal BPE)
 - **Stage 2:** learn "superwords" that span multiple words/whitespace
- **Critique:** Only works for languages for many white spaces (e.g. English). What about Chinese, Japanese and other no-white-space languages?

SuperBPE

POS tag	#	Example Tokens
NN, IN	906	_case_of, _hint_of, _availability_of, _emphasis_on, _distinction_between
VB, DT	566	_reached_a, _discovered_the, _identify_the, _becomes_a, _issued_a
DT, NN	498	_this_month, _no_idea, _the_earth, _the_maximum, _this_stuff
IN, NN	406	_on_top, _by_accident, _in_effect, _for_lunch, _in_front
IN, DT	379	_on_the, _without_a, _alongside_the, _for_each
IN, DT, NN	333	_for_a_living, _by_the_way, _into_the_future, _in_the_midst
NN, IN, DT	270	_position_of_the, _component_of_the, _review_of_the, _example_of_this
IN, DT, JJ	145	_like_any_other, _with_each_other, _for_a_short, _of_the_entire
VB, IN, DT	121	_worked_as_a, _based_on_the, _combined_with_the, _turned_into_a
IN, DT, NN, IN	33	_at_the_time_of, _in_the_presence_of, _in_the_middle_of, _in_a_way_that
,, CC, PRP, VB	20	, _and_it_was, , _but_I_think, , _but_I_have, , _but_I_am
IN, DT, JJ, NN	18	_in_the_long_run, _on_the_other_hand, _for_the_first_time, _in_the_same_way

Table 3: **The most common POS tags for tokens of 2, 3, and 4 words in SuperBPE**, along with random example tokens for each tag. NN = noun, IN = preposition, VB = verb, DT = determiner, CC = conjunction, JJ = adjective, PRP = pronoun.

Lang Tokenization in Visual Space

- Some prior work have tried encoding text as pixels. (Rust et al.)
- Recent works (e.g., DeepSeek-OCR) render a page as an image, and encode it into a small number of "vision tokens".
- They boast 10x compression with minimal accuracy drop.

